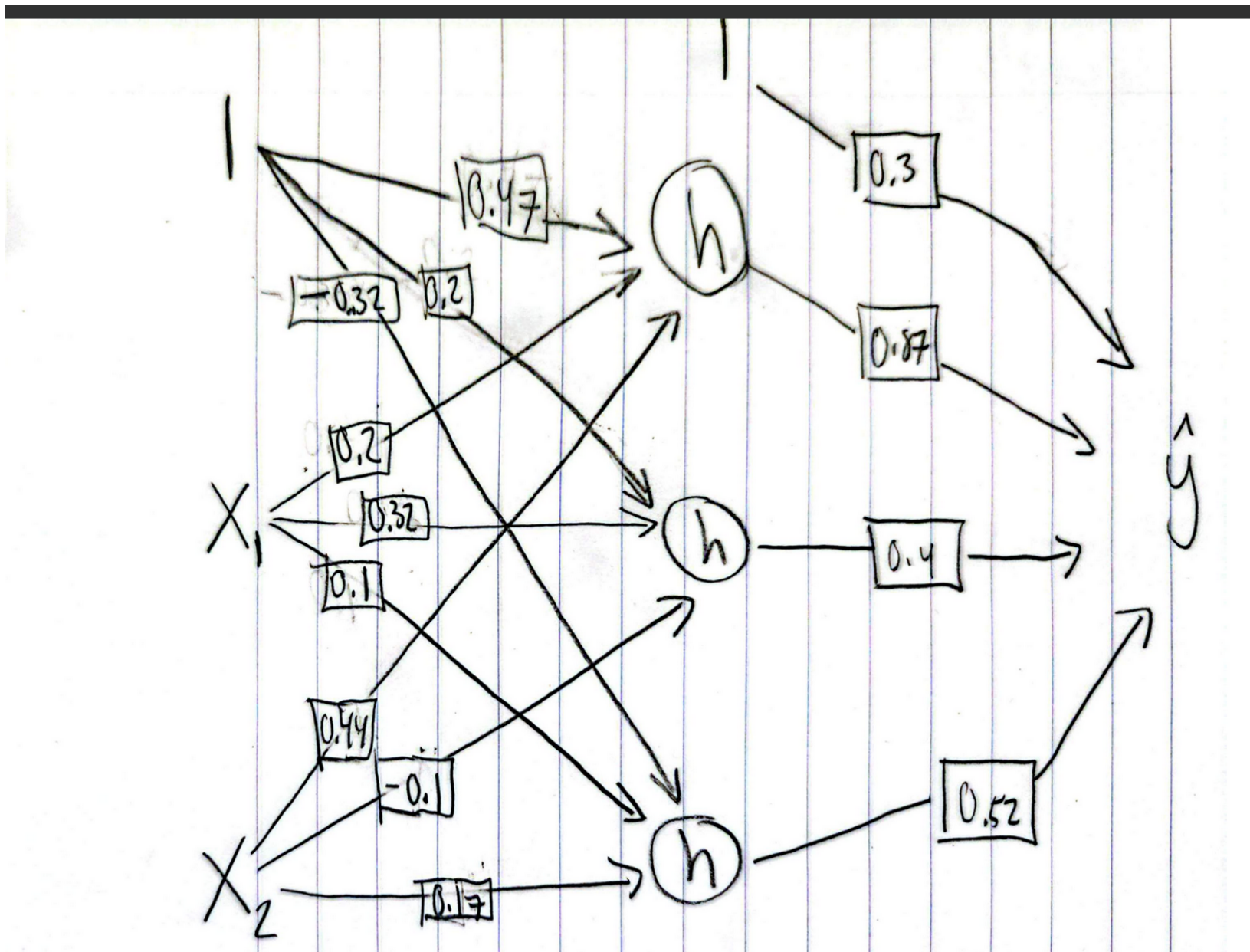


Problem 1

(a)



(b)

```
In [1]: import numpy as np
```

```
@np.vectorize
def h(x):
    if x <= 0:
        return 0
    else:
        return x

@np.vectorize
def h_prime(x):
    if x <= 0:
        return 0
    else:
        return 1

W1 = np.array([
    [0.2, 0.44],
    [0.32, -0.1],
    [0.1, 0.17],
])

b1 = np.array([0.47, 0.2, -0.32])

W2 = np.array([
    [0.87, 0.04, 0.52],
])

b2 = np.array([0.3])

x1 = np.array([-0.14, 0.15])
q1 = h(x1.T@W1.T + b1.T)
print(q1)
print(q1.T@W2.T + b2.T)
```

```
[0.508 0.1402 0.    ]
[0.747568]
```

(c)

In classification, the output of the neural should be of dimension M (the number of classes), which feeds into an additional softmax activation function in order to predict the probabilities of each class. Since the neural net defined above is a single number on the output, instead of M outputs, we know that it cannot possibly be used for classification.

Problem 2

We will follow the steps on page 143 of the textbook

```
In [2]: x = np.array([1, -1])
        y = np.array([0.1])

        q0 = x
        print(f"q0 = x = {x}")
        z1 = W1@q0 + b1
        print(f"z1 = {z1}")
        q1 = h(z1)
        print(f"q1 = {q1}")
        z2 = W2@q1+b2
        print(f"z2 = {z2}")
        J = (y - z2)**2
        print(f"J(theta) = {J}")
```

```
q0 = x = [ 1 -1]
z1 = [ 0.23  0.62 -0.39]
q1 = [0.23 0.62 0. ]
z2 = [0.5249]
J(theta) = [0.18054001]
```

```
In [3]: q0.shape
```

```
Out[3]: (2,)
```

```
In [4]: dz2 = -2*(y - z2)

        dW2 = dz2 @ q1.reshape((len(q1), 1)).T
        db2 = dz2
```

```
dq1 = W2.T@dz2
dz1 = dq1 * h_prime(z1)
dW1 = dz1.reshape((len(dz1), 1)) @ q0.reshape((len(q0), 1)).T
db1 = dz1

print("z2")
print(z2)
print()

print("dz2")
print(dz2)
print()

print("dW2")
print(dW2)
print()

print("db2")
print(db2)
print()

print("dq1")
print(dq1)
print()

print("dz1")
print(dz1)
print()

print("dW1")
print(dW1)
print()

print("db1")
print(db1)
print()
```

```
z2
[0.5249]

dz2
[0.8498]

dW2
[0.195454 0.526876 0.      ]

db2
[0.8498]

dq1
[0.739326 0.033992 0.441896]

dz1
[0.739326 0.033992 0.      ]

dW1
[[ 0.739326 -0.739326]
 [ 0.033992 -0.033992]
 [ 0.        0.        ]]

db1
[0.739326 0.033992 0.      ]
```

```
In [5]: gamma = 0.05
W1_new = W1 - gamma*dW1
b1_new = b1 - gamma*db1

W2_new = W2 - gamma*dW2
b2_new = b2 - gamma*db2

print(f"Updated W1")
print(W1_new)
print()

print("Updated b1")
print(b1)
print()
```

```

print(f"Updated W2")
print(W2_new)
print()

print("Updated b2")
print(b2)
print()

```

Updated W1

```

[[ 0.1630337  0.4769663]
 [ 0.3183004 -0.0983004]
 [ 0.1        0.17       ]]

```

Updated b1

```

[ 0.47  0.2 -0.32]

```

Updated W2

```

[[0.8602273 0.0136562 0.52      ]]

```

Updated b2

```

[0.3]

```

```

In [6]: x = np.array([1, -1])
        y = np.array([0.1])

        q0 = x
        print(f"q0 = x = {x}")
        z1 = W1_new@q0 + b1_new
        print(f"z1 = {z1}")
        q1 = h(z1)
        print(f"q1 = {q1}")
        z2 = W2_new@q1+b2_new
        print(f"z2 = {z2}")
        J = (y - z2)**2
        print(f"J(theta) = {J}")

```

```

q0 = x = [ 1 -1]
z1 = [ 0.1191011  0.6149012 -0.39      ]
q1 = [0.1191011 0.6149012 0.          ]
z2 = [0.36836123]
J(theta) = [0.07201775]

```

```
In [7]: # Q = np.array([
#       q1,
# ])

# Z = np.array([
#       z1.reshape(),
#       np.atleast_1d(z2),
# ])
```

Problem 3

```
In [8]: def compute_pooling_layer_output(input, stride=2, method=np.max):
        output = np.zeros((input.shape[0]//stride, input.shape[1]//stride))
        for i in range(output.shape[0]):
            for j in range(output.shape[1]):
                output[i, j] = method(input[stride*i:stride*(i+1), stride*j:stride*(j+1)])
        return output

pooling_layer_input = np.array([
    [5, 2, 1, 0, 2, 4],
    [2, 3, 0, 3, 4, 2],
    [5, 7, 7, 4, 3, 4],
    [0, 4, 9, 0, 2, 3],
    [1, 3, 5, 2, 6, 2],
    [2, 2, 4, 3, 8, 0]
])

print(f"Using Max Pooling: \n{compute_pooling_layer_output(pooling_layer_input, stride=2, method=np.max)}")
print(f"Using Average Pooling: \n{compute_pooling_layer_output(pooling_layer_input, stride=2, method=np.me
```

Using Max Pooling:

```
[[5. 3. 4.]
 [7. 9. 4.]
 [3. 5. 8.]]
```

Using Average Pooling:

```
[[3.  1.  3. ]
 [4.  5.  3. ]
 [2.  3.5 4. ]]
```


Problem 4

	Convolutional layers			Dense layers	
	Layer 1	Layer 2	Layer 3	Layer 4	Layer 5
Number of filters/output channels	4	8	12	—	—
Filter rows and columns	(5×5)	(5×5)	(4×4)	—	—
Stride	1	2	2	—	—
Number of hidden units	3 136	1 568	588	200	10
Number of parameters (including offset vector)	104	808	1 548	117 800	2 010

Layer 1

```
In [9]: # The weight tensor is of dimensions
# (filter rows × filter columns × input channels × output channels)
filter_rows = 5
filter_columns = 5
input_channels = 1
output_channels = 4

num_weight_parameters = filter_rows*filter_columns*input_channels*output_channels
# The number of parameters corresponds to the number of elements in the
# weight tensor plus the number of elements in the offset vector (there is one element for each output char
num_offset_parameters = output_channels

num_parameters_first_layer = num_weight_parameters + num_offset_parameters
print(f"Number of parameters in first layer: {num_parameters_first_layer}")
```

```
# With a stride of 1, each of the p=784 pixel in the input image will have a corresponding hidden unit in the first layer
p = 28**2
num_hidden_units_first_layer = p * 4
print(f"Number of hidden units in first layer: {num_hidden_units_first_layer}")
```

Number of parameters in first layer: 104

Number of hidden units in first layer: 3136

Layer 2

It is a similar process for the next two layers

```
In [10]: # The weight tensor is of dimensions
# (filter rows × filter columns × input channels × output channels)
filter_rows = 5
filter_columns = 5
input_channels = 4
output_channels = 8

num_weight_parameters = filter_rows*filter_columns*input_channels*output_channels
# The number of parameters corresponds to the number of elements in the
# weight tensor plus the number of elements in the offset vector (there is one element for each output char
num_offset_parameters = output_channels

num_parameters_second_layer = num_weight_parameters + num_offset_parameters
print(f"Number of parameters in second layer: {num_parameters_second_layer}")

p = 28**2

# The filter is now a tensor filter rows × filter columns × input channels
stride = 2
num_elements_in_filter = filter_rows*filter_columns*input_channels

num_hidden_units_in_single_channel_of_previous_layer = num_hidden_units_first_layer/input_channels

# We simply need to divide by the stride along each dimension the stride is applied (2 dimensions),
# Thus we divide by 2^2
num_hidden_units_in_single_channel_of_current_layer = num_hidden_units_in_single_channel_of_previous_layer

num_hidden_units_second_layer = num_hidden_units_in_single_channel_of_current_layer* output_channels
print(f"Number of hidden units in second layer: {num_hidden_units_second_layer}")
```

Number of parameters in second layer: 808

Number of hidden units in second layer: 1568.0

Layer 3

```
In [11]: # The weight tensor is of dimensions
# (filter rows × filter columns × input channels × output channels)
filter_rows = 4
filter_columns = 4
input_channels = 8
output_channels = 12
```

```
num_weight_parameters = filter_rows*filter_columns*input_channels*output_channels
# The number of parameters corresponds to the number of elements in the
# weight tensor plus the number of elements in the offset vector (there is one element for each output char
num_offset_parameters = output_channels

num_parameters_third_layer = num_weight_parameters + num_offset_parameters
print(f"Number of parameters in third layer: {num_parameters_third_layer}")

# The filter is now a tensor filter rows × filter columns × input channels
stride = 2
num_elements_in_filter = filter_rows*filter_columns*input_channels

num_hidden_units_in_single_channel_of_previous_layer = num_hidden_units_second_layer/input_channels

# We simply need to divide by the stride along each dimension the stride is applied (2 dimensions),
# Thus we divide by 2^2
num_hidden_units_in_single_channel_of_current_layer = num_hidden_units_in_single_channel_of_previous_layer

num_hidden_units_third_layer = num_hidden_units_in_single_channel_of_current_layer* output_channels
print(f"Number of hidden units in third layer: {num_hidden_units_third_layer}")
```

Number of parameters in third layer: 1548

Number of hidden units in third layer: 588.0